Analiza Dużych Zbiorów Danych

Wykład 4: Rozproszone przetwarzanie wsadowe: Apache Spark

dr inż. Krzysztof Ropiak
Uniwersytet Warmińsko-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Plan wykładu

- **1. Od MapReduce do In-Memory Computing i Architektura Apache Spark**
  Dlaczego dyskowy MapReduce przegrał z czasem • Rewolucja RDD i odporność na awarie przez *Lineage Graph* • Architektura fizyczna klastra (Driver, Executors, Cluster Manager) • Hierarchia wykonania (Job ➔ Stage ➔ Task) • Ewolucja API • Lazy Evaluation i transformacje *Narrow* vs *Wide*
- **2. Realia produkcyjne: Diagnoza padającego joba (Shuffle, OOM, Data Skew)**
  Studium przypadku "TelecomPol" (syndrom 99% postępu) • Anatomia operacji *Shuffle* • Model *Unified Memory Manager* • Dwa oblicza błędu `OutOfMemoryError` • Skośność danych (*Data Skew*) • Pułapka PySpark UDF vs Apache Arrow • Diagnostyka w **Spark UI**
- **3. Nowoczesne rozwiązanie: Silnik optymalizacyjny pod maską**
  Kompilator zapytań **Catalyst Optimizer** • Silnik wykonawczy **Project Tungsten** (*Off-Heap Memory* i *Whole-Stage CodeGen*) • Rewolucja **Adaptive Query Execution (AQE)** • Techniki *Key Salting*, `repartition` vs `coalesce` oraz `cache()`
- **4. Synteza inżynierska, natywne silniki C++/Rust i dyskusja**
  Zmierzch czystego JVM (Photon, Velox, Comet) • Złota macierz strategii złączeń (Joins) • Dwa produkcyjne scenariusze decyzyjne • Podsumowanie

01. Dlaczego klasyczny MapReduce przegrał z czasem?

W modelu **Hadoop MapReduce** (Wykład 1) każdy etap obliczeń był zmuszony do pełnej materializacji wyników pośrednich na dyskach systemu HDFS z 3-krotną replikacją:

```
Klasyczny Hadoop MapReduce (np. 3-etapowe zapytanie lub algorytm iteracyjny ML / PageRank):
[HDFS Read] ➔ (Map 1 ➔ Reduce 1) ➔ [Zapis HDFS 3x] ➔ [Odczyt HDFS] ➔ (Map 2 ➔ Reduce 2) ➔ [Zapis HDFS 3x]
                                          ▲                  ▲
                      90% czasu zadania to czekanie na dyskowe I/O, sieć i serializację!

Apache Spark (In-Memory DAG Execution):
[Storage Read] ➔ (Etap 1 w RAM) ──────► (Etap 2 w RAM) ──────► (Etap 3 w RAM) ➔ [Wynik końcowy]
                                ▲                  ▲
                  Potokowe przetwarzanie w pamięci operacyjnej (10x – 100x szybciej)
```

Wąskie gardło MapReduce

- Brak możliwości utrzymania danych pomiędzy krokami w pamięci RAM.
- Sztywny dwuetapowy szablon (każdy problem trzeba wyrazić jako jedną parę `Map` + `Reduce`).
- Koszmarna wydajność przy uczeniu maszynowym (np. K-Means, regresja logistyczna) i zapytaniach interaktywnych.

Przełom In-Memory (AMPLab, 2009–2012)

- Keny pamięci RAM spadły na tyle, by klastry mogły dysponować terabajtami pamięci operacyjnej.
- Zamiast pojedynczego kroku Map+Reduce, budujemy pełny acykliczny graf skierowany (**DAG**).
- Dysk i sieć są dotykane tylko wtedy, gdy jest to absolutnie konieczne.

Źródło: M. Zaharia et al., "Spark: Cluster Computing with Working Sets", USENIX HotCloud 2010.

02. Rewolucja RDD: Odporność na awarie bez replikacji w RAM

Jak zapewnić odporność na awarię węzła w pamięci RAM bez kosztownego kopiowania każdego bajtu na 3 serwery (jak w systemach DSM – *Distributed Shared Memory*)?

RDD (Resilient Distributed Dataset)

Abstrakcja podstawowa Apache Spark (NSDI 2012)

- **Resilient (Odporny):** Jeśli serwer zgaśnie i straci zawartość RAM, Spark automatycznie odtworzy brakujący fragment.
- **Distributed (Rozproszony):** Dane są fizycznie podzielone na **partycje** rozsiane po wielu maszynach klastra.
- **Immutable (Niemutowalny):** Raz utworzonego zbioru RDD nie można zmienić w miejscu – można go jedynie przekształcić w nowy RDD.

Sekret: Graf Rodowodu (Lineage Graph)

Przeliczanie zamiast replikacji!

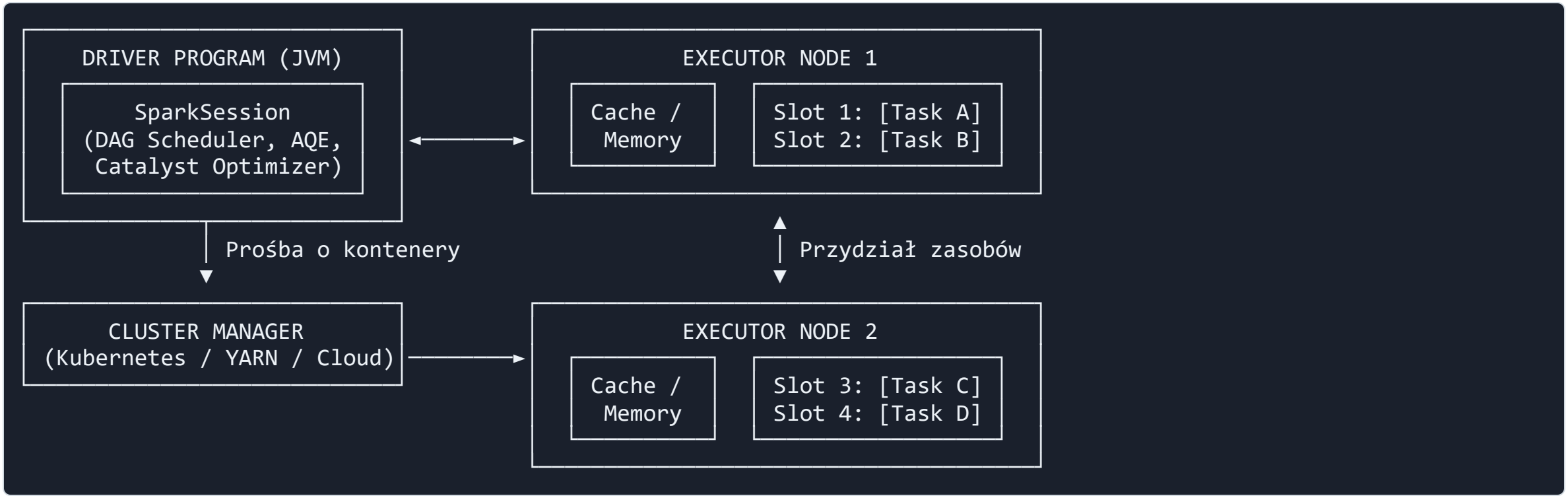
- Ponieważ zbiory są **niemutowalne**, a operacje deterministyczne, Spark nie musi replikować danych pośrednich w RAM.
- Spark pamięta "przepis kucharski" (graf transformacji), jak powstała każda partycja: `RDD_C = RDD_B.filter(...) = RDD_A.map(...)`.
- Gdy węzeł nr 4 ulegnie awarii, Spark odtwarza **tylko utraconą partycję nr 4**, aplikując przepis od najbliższego punktu źródłowego!

Geniusz inżynierski (Best Paper Award NSDI 2012): Zastąpienie fizycznej replikacji pamięci RAM logicznym grafem rodowodu (*Lineage*) pozwoliło zaoszczędzić **3-krotność pamięci operacyjnej** i wyeliminowało ruch sieciowy podczas normalnej pracy bez awarii.

Źródło: M. Zaharia, M. Chowdhury, T. Das et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing", USENIX NSDI 2012.

03. Fizyczna architektura klastra Apache Spark

Każda aplikacja Spark działa w architekturze **Master–Worker**:



03-1. Fizyczna architektura klastra Apache Spark

- **Driver Program:** Pojedynczy proces zarządzający cyklem życia aplikacji. Tłumaczy kod użytkownika (Python/SQL/Scala) na plan fizyczny, dzieli pracę na drobne zadania (*Tasks*) i rozsyła je do wykonania.
- **Cluster Manager:** Zarządca infrastruktury (Kubernetes, Hadoop YARN, Databricks Cloud Manager) – wie tylko, ile jest wolnych procesorów i RAM-u w serwerowni, nie rozumie logiki zapytania SQL.
- **Executors:** Procesy robocze JVM uruchamiane na węzłach klastra. Każdy posiada przydzieloną pamięć RAM oraz pewną liczbę rdzeni procesora (**Cores / Slots**) do równoległego wykonywania zadań.

Źródło: Apache Spark Official Documentation, "Cluster Mode Overview", 2024.

04. Hierarchia egzekucji kodu: Od Aplikacji do Pojedynczego Tasku

W jaki sposób Twoje zapytanie SQL lub skrypt PySpark zamienia się w fizyczne wątki na procesorach w klastrze?

1. Application

Cały program / sesja

- Twój skrypt lub notatnik przypisany do jednej instancji `SparkSession`.
- Może zawierać dziesiątki zapytań i operacji.

➔

2. Job (Zadanie)

1 Action = 1 Job

- Wyzwalany przez każdą **Akcję** (np. `count()`, `save()`, `show()`).
- Posiada własny acykliczny graf wykonania (DAG).

➔

3. Stage (Etap)

Granica wymiany sieciowej

- Zestaw operacji, które można wykonać lokalnie na węźle bez udziału sieci.
- Każdy **Shuffle** tnie Job na osobne Stage'e!

➔

4. Task (Jednostka)

1 Partycja = 1 Task

- Najmniejsza jednostka pracy wysyłana do Executora.
- Wykonywana przez **1 wątek CPU (1 Slot)** na dokładnie **1 partycji danych**.

Złote równanie równoległości w Apache Spark:

`1 Partycja Danych = 1 Task = 1 Wątek Procesora (Core/Slot)`.

Jeśli masz klaster ze 100 rdzeniami CPU, ale Twoje dane są zapisane w 1 niepodzielnym pliku (1 partycja), to **99 rdzeni będzie bezczynnie stało**, a cały plik będzie mielony przez 1 rdzeń!

Źródło: J. S. Damji, B. Wenig, T. Das, D. Lee, "Learning Spark, 2nd Edition", O'Reilly 2020 (Rozdział 2).

05. Ewolucja API w Sparku: Dlaczego deklaratywność wygrała?

Ewolucja Sparka od niskopoziomowych funkcji lambda ku wysokopoziomowym abstrakcjom relacyjnym:

Cechy / Kryterium	RDD API (Spark 1.0, 2014)	DataFrame API / Spark SQL (Spark 1.3 / 2.0+)	Dataset API (Spark 1.6 / 2.0+, tylko Scala/Java)
Poziom abstrakcji	Niskopoziomowy, imperatywny ("*JAK to policzyć*")	Wysokopoziomowy, deklaratywny ("*CO chcę otrzymać*")	Pośredni (łączy zalety RDD i DataFrame)
Znajomość schematu przez silnik	**Brak (Czarna skrzynka)** – obiekty Javy/Pythona są nieprzezroczyste	**Pełna znajomość kolumn i typów** (jak w tabeli relacyjnej)	Pełna znajomość schematu dzięki *Encoders*
Optymalizator zapytań (Catalyst)	❌ Brak (wykonuje kod dokładnie tak, jak napisał programista)	✅ **Tak** (automatycznie przepisuje i optymalizuje plan)	✅ **Tak** (częściowo ograniczony przy funkcjach lambda)
Reprezentacja w pamięci	Obiekty sterty JVM (duży narzut pamięciowy i GC)	**Binarny format Tungsten** (`UnsafeRow`) poza stertą	Konwersja pomiędzy `UnsafeRow` a obiektami JVM
Wydajność między językami	Scala szybsza niż Python (narzut serializacji w Pythonie)	**Identyczna wydajność w Pythonie, SQL, Scali i R!**	Niedostępne w Pythonie ani R

> **Wniosek praktyczny:** W nowoczesnym Sparku (3.x / 4.x) używamy niemal wyłącznie **DataFrame API oraz Spark SQL**. Pisząc w PySparku za pomocą DataFrame API, Twój kod nie wykonuje się w powolnym interpreterze Pythona – Python służy jedynie do zbudowania planu zapytania, który wykonuje natywny silnik JVM/C++!

06. Leniwa ewaluacja (Lazy Evaluation): Transformacje vs Akcje

Operacje w Apache Spark dzielą się rygorystycznie na dwie kategorie:

Transformacje (Transformations)

Operacje LENIWE (Lazy) – zwracają nowy DataFrame

- Przykłady: `select()`, `filter()`, `where()`, `groupBy()`, `join()`, `orderBy()`, `withColumn()`.
- Wywołanie transformacji **NIE przetwarza żadnych danych** i trwa ułamek milisekundy!
- Dopisuje jedynie kolejny krok do logicznego planu wykonania (grafu DAG).

Akcje (Actions)

Operacje ZACHŁANNE (Eager) – wymuszają wynik

- Przykłady: `count()`, `show()`, `collect()`, `take(n)`, `write.parquet()`, `write.saveAsTable()`.
- Wywołanie akcji mówi silnikowi: "*Potrzebuję teraz fizycznego wyniku na ekranie lub na dysku!*".
- Uruchamia kompilator Catalyst, optymalizuje cały łańcuch i wysyła zadania na klaster.

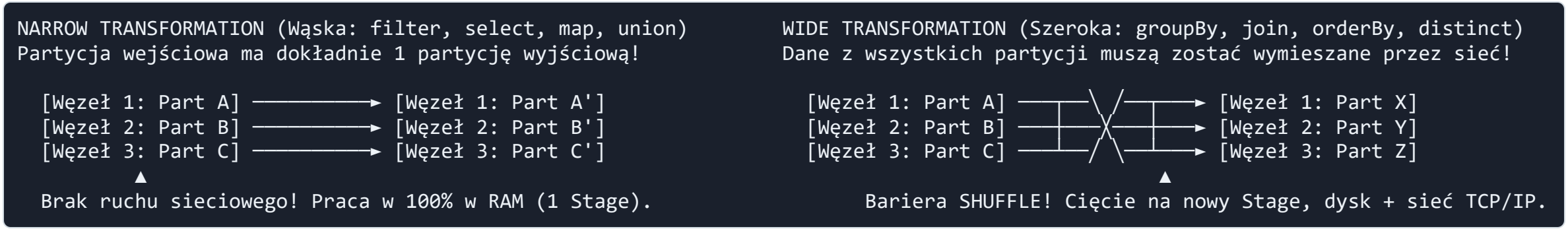
```
# Krok 1-3: Transformacje (czas wykonania: 0.002 sekundy - klaster nic jeszcze nie czyta!)
df_logs   = spark.read.parquet("s3://shop/logs_10TB/")
df_errors = df_logs.filter(df_logs.status == 500)
df_final  = df_errors.select("user_id", "timestamp")

# Krok 4: Akcja (Dopiero TERAZ rusza maszyna! Spark widzi cały plan i czyta z S3 tylko 2 kolumny i bloki ze statusem 500)
df_final.write.format("delta").save("s3://shop/errors_report/")
```

Źródło: Apache Spark Programming Guide, "RDD & DataFrame Operations: Transformations and Actions", 2024.

07. Narrow vs Wide Transformations: Gdzie rodzi się koszt sieciowy?

To najważniejszy podział architektoniczny w całym silniku Spark. Decyduje o tym, czy zadanie wykona się w sekundy, czy zawiesi sieć w serwerowni:



Transformacje Wąskie (Narrow)

- Każdy wiersz można przetworzyć niezależnie, bez wiedzy o wierszach leżących na innych serwerach.
- Spark łączy wiele wąskich operacji w jeden potok (*Pipelining*) wykonywany w pojedynczym przebiegu w pamięci RAM.

Transformacje Szerokie (Wide)

- Wymagają zgrupowania rekordów o tym samym kluczu z całego klastra na jednym węźle (np. `GROUP BY user_id`).
- Wymuszają kosztowną operację **Shuffle** (zapis na dysk + wysyłka przez sieć) i tworzą granicę nowego etapu (**Stage Boundary**).

Źródło: M. Zaharia et al., "Resilient Distributed Datasets", USENIX NSDI 2012.

08. Studium przypadku: Nocny potok "TelecomPol" staje na 99%

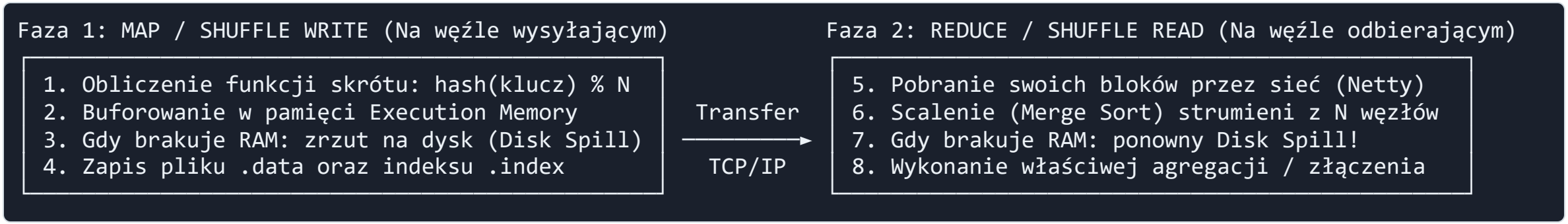
Scenariusz produkcyjny (Awaria o 03:30 w nocy):
Operator telekomunikacyjny "TelecomPol" przetwarza co noc **2 miliardy rekordów bilingowych CDR** (*Call Detail Records* – ok. 800 GB danych Parquet). Zadanie wykonuje złączenie z tabelą klientów i agregację czasu połączeń po numerze telefonu (`GROUP BY msisdn`).

- **Objawy w panelu monitoringu (Spark UI):**
  - Etap (Stage) składa się z **200 zadań (Tasks)**.
  - **199 z 200 zadań** wykonuje się błyskawicznie w ciągu **90 sekund**.
  - Pasek postępu zatrzymuje się na magicznym **99.5% (199/200 Tasks Succeeded)**.
  - Ostatni, 200. task mieli dane samotnie przez **3,5 godziny**, wentylatory jednego serwera pracują na 100%, aż w końcu całe zadanie kończy się błędem:
    `org.apache.spark.SparkException: Job aborted due to stage failure: java.lang.OutOfMemoryError: Java heap space`

Pytanie diagnostyczne: Dlaczego 199 rdzeni skończyło pracę w półtorej minuty, a jeden rdzeń pracował ponad 3 godziny i zabił cały klaster? Aby odpowiedzieć, musimy rozebrać na części pierwsze trzy demony Sparka: **Shuffle**, **Pamięć Executora** oraz **Data Skew**.

09. Anatomia operacji Shuffle: Najdroższy proces w klastrze

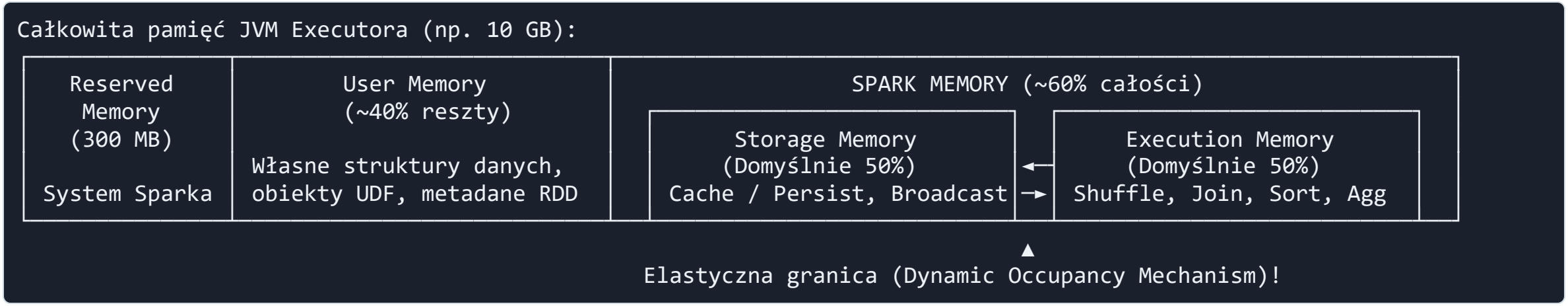
Co naprawdę dzieje się pod maską, gdy piszesz niewinnie wyglądające `df.groupBy("klucz").sum("kwota")` lub `df1.join(df2, "id")`?



- **Dlaczego Shuffle jest tak kosztowny? Kumuluje 4 najcięższe operacje w informatyce:**
  i. **CPU:** Serializacja obiektów do strumienia bajtów, liczenie funkcji skrótu (*MurmurHash3*), sortowanie i kompresja.
  ii. **Disk I/O:** Spark **zawsze** zapisuje pliki Shuffle na lokalny dysk węzła (*Shuffle Write*), aby w razie awarii odbiorcy nie przeliczać poprzedniego etapu od nowa!
  iii. **Network I/O:** Wysłanie gigabajtów/terabajtów danych pomiędzy wszystkimi węzłami klastra (*All-to-All communication*).
  iv. **Memory Pressure:** Konieczność utrzymania tablic mieszających lub buforów sortujących w pamięci RAM odbiorcy.

10. Zarządzanie pamięcią wewnątrz Executora (Unified Memory Manager)

Pamięć RAM przydzielona pojedynczemu Executorowi (np. `spark.executor.memory = 10 GB`) nie jest jednym wielkim workiem. Od wersji Spark 1.6 zarządza nią **Unified Memory Manager**:



- **Elastyczna granica pomiędzy *Storage* a *Execution***:
  - Jeśli nie używasz `df.cache()`, **Execution Memory** może zająć 100% obszaru *Spark Memory* na potrzeby wielkich złączeń i sortowania.
  - Jeśli używasz `cache()`, ale nagle rusza ciężki `JOIN`, *Execution Memory* **wyrzuci (evict)** zbuforowane bloki *Storage* na dysk, by ratować wykonanie zapytania!
  - **Kiedy dochodzi do Disk Spill?** Gdy sam obszar *Execution Memory* wypełni się po brzegi – Spark zrzuca nadmiar tabel sortowania na dysk SSD, co chroni przed OOM, ale **spowalnia zadanie 10-krotnie**.

11. Dwa oblicza katastrofy `OutOfMemoryError` (OOM)

Kiedy widzisz w logach błąd `OutOfMemoryError`, pierwsze pytanie inżyniera brzmi: **Kto wybuchł – Driver czy Executor?** Przyczyny i lekarstwa są zupełnie inne!

1. Driver OOM (Wybucha mózg klastra)

Przepełnienie pamięci na węźle zarządzającym

- **Grzech główny #1:** Wywołanie `df.collect()` lub `df.toPandas()` na dużym zbiorze – próba ściągnięcia 500 GB danych z klastra do pamięci jednego procesu Drivera!
- **Grzech główny #2:** Zbyt duży `BroadcastJoin` – Driver najpierw sam pobiera całą rozgłaszaną tabelę do swojej pamięci, aby zbudować z niej Hash Mapę.
- **Lekarstwo:** Zakaz używania `collect()` na danych produkcyjnych (używaj `take(20)` lub zapisu do S3), zmniejszenie progu `autoBroadcastJoinThreshold`.

2. Executor OOM (Wybucha węzeł roboczy)

Przepełnienie pamięci podczas przetwarzania partycji

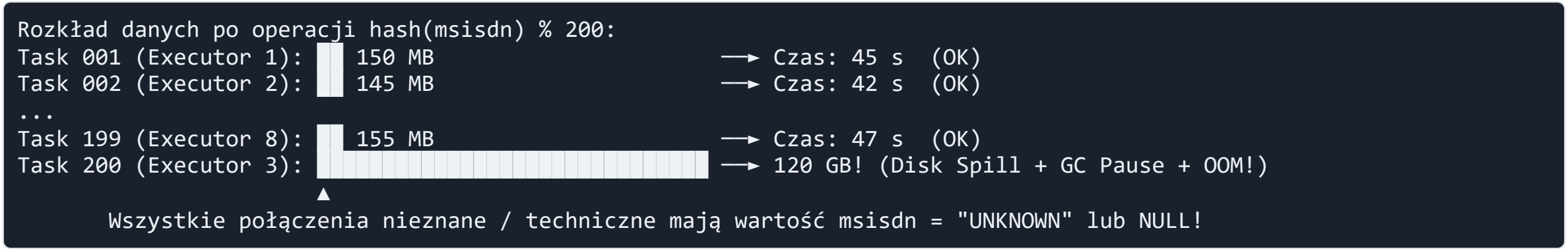
- **Przyczyna #1 (Data Skew):** Jedna gigantyczna partycja (np. 40 GB dla klucza `NULL`) trafia do jednego Tasku i nie mieści się w *Execution Memory*.
- **Przyczyna #2 (Zbyt mało partycji):** Czytanie lub tasowanie 1 TB danych podzielonego tylko na 10 partycji (100 GB na jeden wątek!).
- **Przyczyna #3 (Za dużo rdzeni na Executor):** 16 wątków w jednym Executorze walczy o wspólną pulę RAM w tym samym ułamku sekundy.

Pułapka kontenerowa (Kubernetes / YARN): Czasami nie zobaczysz błędu Java OOM, lecz kod wyjścia `Exit Code 137 (OOMKilled)`. Oznacza to, że proces Pythona lub pamięć *Off-Heap* przekroczyła limit kontenera Linuxa (`spark.executor.memoryOverhead`) i jądro systemu bezlitośnie zabiło proces sygnałem `SIGKILL`!

Źródło: Databricks Documentation, "Debugging OutOfMemoryError Exceptions in Apache Spark", 2023.

12. Zjawisko skośności danych (Data Skew) i problem maruderów

Wracamy do awarii w "TelecomPol" (200 tasków). To podręcznikowy przykład **Data Skew**:



- **Dlaczego funkcja skrótu zawiodła?**
  - Funkcja `hash(klucz)` jest deterministyczna: ten sam klucz --> **zawsze ten sam numer partycji**!
  - W realnym świecie dane nigdy nie mają rozkładu jednostajnego – rządzi nimi **rozkład potęgowy (Prawo Zipfa / Zasada Pareto)**:
    - E-commerce: 1 konto hurtownika lub bota generuje 20% wszystkich zamówień.
    - Telekomunikacja / IoT: uszkodzone czujniki lub połączenia nieautoryzowane wysyłają milion rekordów z kluczem `NULL` lub `-1`.
  - W efekcie **czas wykonania całego etapu (Stage) jest równy czasowi wykonania najwolniejszego zadania (*Straggler Task*)**.

13. Pułapka klasycznych funkcji UDF w PySparku vs Apache Arrow

Programiści Pythona uwielbiają pisać własne funkcje i rejestrować je jako `@udf`. W klasycznym PySparku to **wydajnościowe samobójstwo**:

```
Klasyczny Python UDF (Wiersz po wierszu – katastrofa wydajności):
[Executor JVM: UnsafeRow] ──(1. Deserializacja i wysyłka przez socket TCP/Pipe: Pickle)──► [Proces Python Worker]
[Executor JVM: UnsafeRow] ◄──(3. Odbiór i ponowna serializacja do formatu JVM: Pickle)─── [2. Pętla for w Pythonie]

Wektoryzowany Pandas UDF (Spark 2.3+ / 3.x z wykorzystaniem Apache Arrow):
[Executor JVM: Kolumny]   ═══(Bezpośredni transfer bloków kolumnowych Arrow / Zero-Copy)═══► [Wektory C/NumPy/Pandas]
```

Dlaczego klasyczny Python UDF zabija klaster?

- **Czarna skrzynka dla Catalysta:** Optymalizator nie wie, co robi kod Pythona – wyłącza wypychanie predykatów i generowanie kodu.
- **Koszt podwójnej serializacji:** Każdy pojedynczy wiersz musi opuścić maszynę JVM, zostać spakowany protokołem `Pickle`, przesłany lokalnym gniazdem do procesu Pythona i wrócić z powrotem!
- Spowolnienie rzędu **10x – 100x** względem wbudowanych funkcji Spark SQL.

Hierarchia dobrych praktyk w PySparku

- **Wybór nr 1 (Zawsze pierwszy!):** Wbudowane funkcje `pyspark.sql.functions` (np. `F.when`, `F.regexp_extract`) – działają w 100% wewnątrz JVM/C++.
- **Wybór nr 2 (Gdy brakuje wbudowanej): Pandas UDF (Vectorized UDF)** – wykorzystuje poznany na Wykładzie 3 format **Apache Arrow** do przesyłania całych paczek kolumn (np. po 10 000 wierszy) i liczy je wektorowo w C.

Źródło: B. Cutler, L. Jin (IBM / Databricks), "Introducing Pandas UDF for PySpark Powered by Apache Arrow", 2017.

14. Rentgen inżyniera danych: Jak czytać Spark UI w 60 sekund?

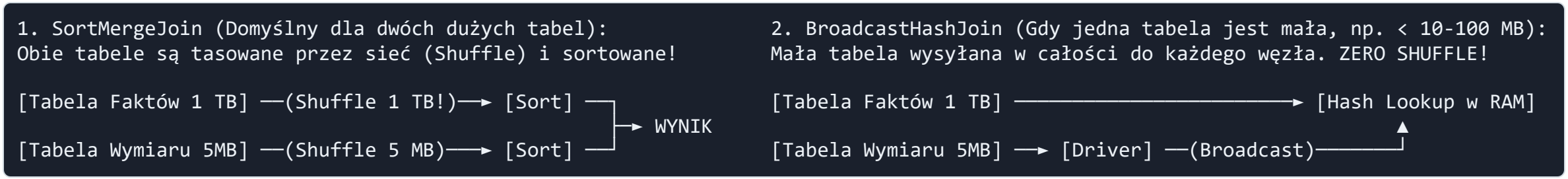
Gdy zadanie działa wolno, profesjonalny inżynier nie zgaduje – otwiera zakładkę **Stages** w **Spark UI** i patrzy na tabelę *Summary Metrics for Completed Tasks*:

Metryka w Spark UI	Co oznacza zdrowy klaster?	Alarm produkcyjny (Co oznacza patologia?)	Diagnoza i działanie naprawcze
Duration (Min / Median / Max)	Min: `18s` | Median: `21s` | Max: `25s` (wartości zbliżone)	Min: `4s` | Median: `12s` | **Max:** `2.4 h`	**Data Skew!** Jeden task dostał gigantyczną ilość danych. Włącz AQE Skew Join lub zastosuj *Salting*.
Shuffle Spill (Memory / Disk)	`0 B / 0 B` (wszystko zmieściło się w RAM)	Memory: `85 GB` | **Disk:** `18 GB`	**Za mało Execution Memory na task!** Zwiększ liczbę partycji (`spark.sql.shuffle.partitions`) lub RAM.
GC Time (Garbage Collection)	`< 2–3%` całkowitego czasu trwania tasku	**> 15–20%** czasu trwania tasku (pasek świeci na czerwono)	Tworzysz miliony obiektów na stercie JVM (np. w RDD lub UDF). Przejdź na natywne DataFrame API (Tungsten).
Scheduler Delay / Task Deserialization	`< 50 ms`	Kilka sekund na każdy krótki task	**Problem małych plików** lub przeładowany Driver wysyłający tysiące mikro-tasków. Wykonaj `OPTIMIZE` .

> **Wskazówka diagnostyczna:** Zawsze porównuj kolumnę **Median (50th percentile)** z kolumną **Max**. Jeśli mediana wynosi 10 sekund i 100 MB, a wartość maksymalna wynosi 45 minut i 30 GB – masz 100% dowód na skośność klucza (*Data Skew*).

15. Matematyka katastrofy złączeń: `SortMergeJoin` vs `BroadcastHashJoin`

Złączenie (`JOIN`) dwóch tabel to najczęstsza operacja w hurtowniach i Lakehouse'ach. Spark posiada dwa zupełnie różne sposoby jej wykonania:



SortMergeJoin (SMJ)

- Wymaga przetasowania (**Shuffle**) oraz fizycznego **posortowania** obu łączonych zbiorów po kluczu złączenia.
- Niezwykle odporny i skalowalny (radzi sobie nawet gdy tabele mają po 100 TB, bo może zrzucać bloki na dysk).
- **Bardzo kosztowny sieciowo i procesorowo**, jeśli jedna z tabel jest małym słownikiem!

BroadcastHashJoin (BHJ)

- Driver pobiera małą tabelę wymiaru (np. słownik krajów lub sklepów) i rozsyła jej kopię do pamięci RAM każdego Executora.
- Wielka tabela faktów (1 TB) **nie rusza się z miejsca!** Złączenie staje się superszybką transformacją wąską (*Narrow*).
- Kontrolowane parametrem `spark.sql.autoBroadcastJoinThreshold` (domyślnie 10 MB, często podnoszone do 50–100 MB).

Źródło: M. Armbrust et al., "Spark SQL: Relational Data Processing in Spark", ACM SIGMOD 2015.

16. Mózg Sparka – Catalyst Optimizer: Cztery fazy kompilacji

Przechodzimy do Modułu 3: Jak Spark potrafi automatycznie zoptymalizować nasze zapytanie? Odpowiada za to regułowo-kosztowy kompilator **Catalyst Optimizer**:

1. Analysis

Unresolved ➔ Resolved Plan

- Sprawdza w **Catalogu** (np. Delta/Iceberg/Unity), czy tabele i kolumny w ogóle istnieją.
- Rozwiązuje typy danych i nazwy atrybutów.

➔

2. Logical Optimization

Regułowa przebudowa drzewa

- Stosuje ponad 80 reguł algebraicznych (*Rule-Based Optimization*).
- Wypycha filtry w dół, usuwa zbędne kolumny, upraszcza stałe.

➔

3. Physical Planning

Co ➔ JAK to policzyć

- Generuje możliwe plany fizyczne.
- Model kosztowy (**CBO**) wybiera np. `BroadcastHashJoin` zamiast `SortMergeJoin`.

➔

4. Code Generation

Kompilacja Janino (Tungsten)

- Tłumaczy wybrany plan fizyczny na czysty bajtkod maszyny JVM w locie.
- Gotowy kod trafia na Executory.

Jak podejrzeć pracę Catalysta w kodzie?
Wystarczy wywołać `df.explain(extended=True)` lub `df.explain("formatted")`! Spark wypisze na ekran wszystkie cztery drzewa transformacji: od tego, co napisałeś, aż po fizyczny plan, który zostanie uruchomiony na klastrze.

Źródło: M. Armbrust, R. S. Xin et al., "Spark SQL: Relational Data Processing in Spark", ACM SIGMOD 2015.

17. Optymalizacje logiczne Catalysta w praktyce

Zobaczmy, jak **Catalyst Optimizer** ratuje nieoptymalnie napisane zapytanie analityka, przebudowując drzewo algebraiczne przed dotknięciem dysku:

```
Zapytanie napisane przez użytkownika (Najpierw wielki JOIN, na końcu filtr i wybór 2 kolumn):
SELECT u.kraj, t.kwota FROM transakcje t JOIN uzytkownicy u ON t.user_id = u.id WHERE t.rok = 2025 AND u.aktywny = true

Plan Naiwny (Gdyby wykonać dosłownie – katastrofa!):          Plan Zoptymalizowany przez Catalyst (Wykonywany w rzeczywistości):
        [Projekcja: kraj, kwota]                                                [Projekcja: kraj, kwota]
                    ▲                                                                       ▲
     [Filtr: rok=2025 AND aktywny=true]                                          [JOIN po user_id = id]
                    ▲                                                                ▲              ▲
   [JOIN całych tabel ze wszystkimi kolumnami!]                    [Projekcja: id, kwota]  [Projekcja: id, kraj]
          ▲                          ▲                                      ▲                       ▲
[Skan: 50 kolumn, 10 lat]  [Skan: 30 kolumn, wszyscy]          [Filtr: rok=2025]          [Filtr: aktywny=true]
                                                                           ▲                       ▲
                                                               [Skan Parquet/Delta]    [Skan Parquet/Delta]
```

17-01. Optymalizacje logiczne Catalysta w praktyce

- **Trzy kluczowe reguły optymalizacji logicznej:**
  i. **Predicate Pushdown (Wypychanie predykatów):** Filtry `WHERE` są spychane na sam dół drzewa – aż do stopki pliku **Apache Parquet** (Wykład 3!), dzięki czemu niepasujące bloki nie są w ogóle czytane z S3.
  ii. **Column Pruning (Przycinanie kolumn):** Catalyst wykrywa, że z 80 kolumn obu tabel potrzebne są tylko 4 (`user_id` , `id` , `kraj` , `kwota` , `rok` , `aktywny`) i wyrzuca resztę na etapie odczytu.
  iii. **Constant Folding & Boolean Simplification:** Wyrażenia takie jak `WHERE kwota > 100 * 12 AND true` są wyliczane raz podczas kompilacji planu (`kwota > 1200`), a nie miliard razy dla każdego wiersza!

Źródło: Apache Spark Catalyst Optimizer Rules (`org.apache.spark.sql.catalyst.optimizer`).

18. Silnik fizyczny – Project Tungsten: Ucieczka ze sterty JVM

W 2015 roku twórcy Sparka zauważyli, że przy szybkich dyskach SSD i sieciach 10/40 GbE wąskim gardłem przestało być I/O, a stał się **narzut samej maszyny wirtualnej Javy (JVM)**:

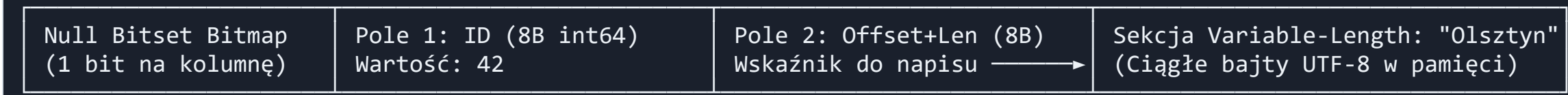
Koszmar obiektów na stercie JVM (Heap)

- Zwykły napis 4-znakowy `"ABCD"` w UTF-8 to **4 bajty** danych.
- Jako obiekt `java.lang.String` na stercie JVM zajmuje aż **48 bajtów** (nagłówek obiektu 64-bit, wskaźniki, tablica znaków, wyrównanie pamięci)!
- Miliony małych obiektów powodują gigantyczne pauzy odśmiecacza pamięci (**JVM Garbage Collector Stop-the-World**).

Rozwiązanie Tungsten: Format `UnsafeRow`

- **Zarządzanie pamięcią Off-Heap (C-style):** Spark sam alokuje ciągłe bloki pamięci binarnej za pomocą `sun.misc.Unsafe`, poza kontrolą Garbage Collectora.
- **Zwarta struktura bitowa:** Wiersz `UnsafeRow` składa się z maski bitowej dla wartości `NULL`, bloków stałej szerokości (8 bajtów na pole) oraz sekcji zmiennej długości.
- Sortowanie i porównywanie wierszy odbywa się na surowych bajtach za pomocą wskaźników pamięciowych!

Układ binarny pojedynczego wiersza w Project Tungsten (UnsafeRow – zwarty ciąg bajtów bez obiektów Javy):



Źródło: R. Xin, J. Rosen (Databricks), "Project Tungsten: Bringing Apache Spark Closer to Bare Metal", 2015.

19. Whole-Stage Code Generation: Kompilacja do jednej pętli CPU

Tradycyjne silniki bazodanowe wykonywały zapytania w tzw. modelu **Volcano Iterator Model** (każdy operator wywołuje wirtualną metodę `.next()` operatora poniżej dla każdego wiersza). **Tungsten Faza 2** (Spark 2.0) zastąpił to kompilacją w locie:

```
Tradycyjny Model Volcano (Miliony wywołań wirtualnych):
  [Project.next()] ◄── [Filter.next()] ◄── [Scan.next()]
         │                    │                   │
   Skoki po pamięci, chybienia Cache L1 i brak optymalizacji
   przez kompilator JIT dla miliarda wierszy!
```

```
Whole-Stage Code Generation (Jedna płaska pętla w bajtkodzie):
  for (row in parquetBatch) {
    if (row.rok == 2025) {        // Bez wywołań funkcji!
    sum += row.kwota * 1.23;      // Dane prosto z rejestrów CPU!
  }
  }
```

19-01. Whole-Stage Code Generation: Kompilacja do jednej pętli CPU

- **Dlaczego *Whole-Stage Code Generation* przyspieszyło Sparka nawet 10-krotnie?**
  - Zamiast przekazywać wiersze przez łańcuch obiektów (`Scan ➔ Filter ➔ Project ➔ Aggregate`), Spark używa mikro-kompilatora **Janino**, aby wygenerować **jedną szytą na miarę pętlę `for` w czystej Javie** dla całego etapu (Stage).
  - **Zero wywołań funkcji wirtualnych** (*Virtual Function Dispatch*).
  - Zmienne pośrednie trzymane są bezpośrednio w ultraszybkich **rejestrach procesora CPU**, zamiast być zapisywane i odczytywane z pamięci RAM.

> **Jak rozpoznać CodeGen w Spark UI?** W planie zapytania zobaczysz gwiazdkę przy nazwie operatora, np. `*(1) Filter` oraz `*(1) Project` – oznacza to, że zostały one sklejone w jedną skompilowaną funkcję natywną `WholeStageCodegen (1)`.

Źródło: T. Neumann, "Efficiently Compiling Efficient Query Plans for Modern Hardware", VLDB 2011.

20. Przełom w Spark 3.x: Adaptive Query Execution (AQE)

Jaka była największa wada **Catalyst Optimizera** w Sparku 2.x? Podejmował wszystkie decyzje **w ciemno przed startem zadania**, bazując często na nieaktualnych lub brakujących statystykach tabel!

```
Klasyczny Spark 2.x (Statyczny plan wykonania):
[Plan fizyczny ustalony przed startem] ──────► [Stage 1] ──────► [Stage 2] ──────► [Stage 3] (Nawet jeśli po drodze filtr odrzucił 99.9% danych!)

Spark 3.x+ z Adaptive Query Execution (AQE):
[Plan początkowy] ──► [Stage 1] ──► (STOP! Odczyt prawdziwych rozmiarów plików Shuffle!) ──► [Re-optymalizacja planu dla Stage 2!]
```

- **Dlaczego granica *Shuffle* jest idealnym momentem na korektę planu?**
  - Przed uruchomieniem zapytania Spark nie wie, ile wierszy przeżyje skomplikowany filtr `WHERE` albo regex.
  - Jednak każdy etap kończący się operacją **Shuffle** zapisuje pliki pośrednie na dysk wraz z dokładnymi statystykami co do bajta!
  - **AQE** zagląda do tych statystyk w trakcie trwania zadania i **przebudowuje dalszą część drzewa wykonania w locie**.

Źródło: W. Fan, H. van Hövell, M. Xue (Databricks), "Adaptive Query Execution: Speeding Up Spark SQL at Runtime", 2020.

21. Trzy supermoce AQE w walce z patologiami produkcyjnymi

Od wersji Spark 3.2 mechanizm **AQE jest włączony domyślnie** (`spark.sql.adaptive.enabled = true`) i automatycznie rozwiązuje trzy klasyczne problemy inżynierskie:

1. Scalanie małych partycji

Coalescing Post-Shuffle Partitions

- Domyślnie po Shuffle Spark tworzy **200 partycji** (`spark.sql.shuffle.partitions=200`).
- Jeśli po filtrze zostało tylko 20 MB danych, 200 partycji po 100 KB tworzy problem małych plików.
- **AQE automatycznie skleja** małe sąsiednie partycje w większe bloki (np. po 64 MB)!

2. Dynamiczny Broadcast Join

Converting SortMergeJoin to BHJ

- Tabela na dysku ma 500 GB, więc Catalyst planuje ciężki `SortMergeJoin` .
- W trakcie Stage 1 warunek `WHERE` wycina 99.9% wierszy i z 500 GB zostaje **tylko 8 MB**!
- **AQE w locie anuluje Shuffle** drugiej tabeli i zamienia złączenie na błyskawiczny `BroadcastHashJoin` !

3. Automatyczny Skew Join

Dynamically Optimizing Skew Joins

- AQE bada rozkład rozmiarów partycji po zapisie Shuffle.
- Gdy wykryje partycję znacznie większą od mediany (domyślnie > 5x mediana i > 256 MB), oznacza ją jako **skośną (Skewed)**.
- **AQE automatycznie tnie gigantyczną partycję** na mniejsze kawałki i duplikuje odpowiadający klucz z drugiej tabeli!

Wracając do awarii w "TelecomPol": Włączenie `spark.sql.adaptive.skewJoin.enabled = true` sprawia, że przy złączeniu gigantyczna partycja nr 200 zostaje automatycznie pocięta na kilkadziesiąt mniejszych zadań i rozdzielona na wszystkie wolne rdzenie klastra!

Źródło: Apache Spark 3.x SQL Performance Tuning Guide – "Adaptive Query Execution", 2024.

22. Ręczne zwalczanie ekstremalnego Data Skew: Solenie kluczy i izolacja NULL

Kiedy automatyczny AQE nie wystarcza (np. przy ciężkich agregacjach `GROUP BY` z funkcją `COUNT(DISTINCT)` lub oknach analitycznych `WINDOW`), inżynier stosuje dwie techniki:

Technika 1: Odfiltrowanie i izolacja kluczy `NULL`

Gdy gorącym kluczem jest brak danych

- W 80% przypadków produkcyjnych przyczyną skosu przy złączeniu jest klucz `NULL`, `""` lub `-1`.
- W SQL warunek `NULL = NULL` jest zawsze fałszywy – rekordy z `NULL` nigdy się nie złączą, a mimo to Spark przesyła je przez sieć do jednego workera!
- **Rozwiązanie:** Podziel DataFrame na dwie części: `df_valid = df.filter("id IS NOT NULL")` (idzie do JOIN) oraz `df_nulls` (doklejany na końcu tanim `unionByName` bez Shuffle!).

Technika 2: Solenie kluczy (Key Salting)

Sztuczne rozproszenie gorącego klucza na N wiader

- Doklejamy do klucza losową liczbę z przedziału `0..N-1` (np. zamiast `"PL"` tworzymy 20 kluczy: `"PL_0"`, `"PL_1"`, ..., `"PL_19"`).
- przy **Agregacji (GROUP BY):** Wykonujemy 2-etapową agregację – najpierw częściową po kluczu "posolonym" (20 rdzeni liczy równolegle), a potem końcową po kluczu oryginalnym.
- Przy **Złączeniu (JOIN):** Dużą tabelę solimy losową liczbą `0..19`, a w drugiej tabeli powielamy (`explode`) gorący klucz 20 razy.

```
Przykład dwuetapowej agregacji z soleniem klucza (Eliminacja wąskiego gardła dla gorącego klucza "HOT_KEY"):
[100 mln wierszy "HOT_KEY"] ──(Dodaj losowy salt 0..9)──► 10 Tasków liczy SUM po "HOT_KEY_0".."HOT_KEY_9" ──► 1 Task sumuje 10 wyników!
```

Źródło: H. Karau, R. Warren, "High Performance Spark", O'Reilly 2017; J. S. Damji et al., "Learning Spark", O'Reilly 2020.

23. Kontrola partycji i pamięci podręcznej: `repartition` vs `coalesce` i `cache`

Świadome zarządzanie liczbą partycji oraz cyklem życia pamięci podręcznej odróżnia architekta od początkującego programisty:

Operacja / Metoda	Co robi pod maską?	Czy wywołuje Shuffle?	Kiedy stosować w praktyce?
`df.repartition(N)` lub `repartition(N, "kol")`	Dzieli lub scala dane w dokładnie `N` równych partycji za pomocą `RoundRobin` lub `HashPartitioning`.	✅ **TAK (Pełny Shuffle przez sieć!)**	Gdy musisz **zwiększyć** równoległość, wyrównać skośne partycje lub przygotować układ plików pod częsty filtr/złączenie po kolumnie `"kol"`.
`df.coalesce(N)`	Łączy istniejące partycje lokalnie na tych samych węzłach bez ruszania danych przez sieć.	❌ **NIE (Narrow Transformation!)**	Wyłącznie gdy chcesz **zmniejszyć** liczbę partycji (np. po złożonym `filter()`, tuż przed zapisem do Parquet/Delta, aby uniknąć małych plików).

23-01. Kontrola partycji i pamięci podręcznej: `repartition` vs `coalesce` i `cache`

> **Pułapka nadużywania `df.cache()` / `df.persist()`:**
> 1. **Zasada jednokrotnego użycia:** Jeśli DataFrame jest używany w kodzie tylko raz (jeden liniowy potok), wywołanie `cache()` **spowolni zadanie** i zmarnuje RAM!
> 2. **Kiedy używać Cache?** Tylko wtedy, gdy z tego samego kosztownie wyliczonego DataFrame'u odgałęziasz **wiele różnych akcji** (np. zapis do 3 tabel lub pętla trenowania modelu ML). Po zakończeniu pracy zawsze zwolnij pamięć metodą `df.unpersist()`!

Źródło: Apache Spark SQL Tuning Guide & RDD Persistence Levels (`StorageLevel.MEMORY_AND_DISK`), 2024.

24. Zmierzch ery czystego JVM: Natywne silniki wektoryzowane (C++ / Rust)

Przez dekadę kod wykonywał się na maszynie wirtualnej Javy (JVM). Jednak w latach 2021–2025 nastąpiła kolejna rewolucja sprzętowa: **zastąpienie warstwy wykonawczej JVM natywnymi silnikami wektoryzowanymi pisanymi w C++ oraz Rust**:

```
Tradycyjny Apache Spark (JVM + Tungsten):
[Kod PySpark / SQL] ──► [Catalyst Optimizer] ──► [Plan Fizyczny Spark] ──► [Wykonywanie w JVM (Whole-Stage CodeGen)]

Nowoczesny Stos Hybrydowy (Spark API + Natywny Silnik Wektoryzowany C++/Rust):
[Kod PySpark / SQL] ──► [Catalyst Optimizer] ──► [Plan Fizyczny] ──► [Silnik C++/Rust: Photon / Velox / DataFusion Comet]
                                                                                ▲
                                                                  100% wektoryzacja SIMD (AVX-512), układ kolumnowy Arrow, zero GC!
```

Databricks Photon (C++)

Nagroda SIGMOD 2022

- Przepisany od zera w C++ wektoryzowany silnik wykonawczy dla platformy Databricks.
- W 100% kompatybilny z API Sparka – nie wymaga zmiany ani jednej linijki kodu!
- Przyspiesza zapytania SQL i zapisy Delta Lake średnio **3x – 8x**.

Meta Velox & Gluten (C++)

Open Source (Meta / Intel)

- Otwartoźródłowa biblioteka wektoryzowanych operatorów C++ stworzona przez inżynierów Meta (VLDB 2022).
- Projekt **Apache Gluten** pozwala podpiąć silnik Velox bezpośrednio pod klaster Apache Spark Open Source.

Apache DataFusion Comet (Rust)

Akcelerator ASF w języku Rust

- Oficjalny subprojekt Apache DataFusion oparty natywnie na formacie **Apache Arrow**.
- Przejmuje wykonanie planu fizycznego Sparka w bezpiecznym pamięciowo kodzie Rust z instrukcjami SIMD.

Źródło: A. Behm et al., "Photon: A Fast Query Engine for Lakehouse Systems", ACM SIGMOD 2022; P. Pedreira et al., "Velox: Meta's Unified Execution Engine", VLDB 2022.

25. Złota macierz strategii złączeń (Joins) w Apache Spark

Strategie fizyczne, jakich używa Spark do łączenia zbiorów danych, wraz z ich kosztami i ograniczeniami:

Strategia fizyczna w Planie	Wymagany Shuffle?	Wymagane Sortowanie?	Obsługiwane warunki złączenia	Kiedy Spark jej używa i jakie niesie ryzyko?
BroadcastHashJoin (BHJ)	❌ **Brak** (Tylko rozgłoszenie małej tabeli)	❌ Brak	Równościowe (`=`)	**Najszybsza strategia!** Gdy jedna tabela jest mniejsza niż próg (np. `< 10-100 MB`). **Ryzyko:** Driver/Executor OOM przy zbyt dużej tabeli.
ShuffleHashJoin (SHJ)	✅ **Tak** (Obie tabele po kluczu)	❌ Brak	Równościowe (`=`)	Gdy jedna tabela jest wyraźnie mniejsza od drugiej, ale za duża na Broadcast. Buduje tablicę Hash per partycja bez kosztu sortowania.
SortMergeJoin (SMJ)	✅ **Tak** (Obie tabele po kluczu)	✅ **Tak** (Obie tabele)	Równościowe (`=`)	**Domyślny koń roboczy dla dużych tabel (TB/PB).** Bardzo stabilny pamięciowo (może robić *Spill* na dysk), ale wolniejszy przez sortowanie.

25-01. Złota macierz strategii złączeń (Joins) w Apache Spark

(ciąg dalszy)

Strategia fizyczna w Planie	Wymagany Shuffle?	Wymagane Sortowanie?	Obsługiwane warunki złączenia	Kiedy Spark jej używa i jakie niesie ryzyko?
BroadcastNestedLoopJoin / CartesianProduct	❌ / ✅	❌ Brak	**Nierównościowe** (`<`, `>`, `BETWEEN`, brak `ON`)	**EKSTREMALNIE NIEBEZPIECZNE!** Złożoność $O(N \times M)$. Złączenie 1 mln wierszy z 1 mln wierszy daje **1 bilion porównań** i zawiesza klaster!

Wskazówka inżynierska (Join Hints): Jeśli wiesz na 100%, że tabela słownikowa po odfiltrowaniu jest mała, możesz wymusić strategię wskazówką: `df_duzy.join(F.broadcast(df_maly), "id")` lub w SQL: `SELECT /*+ BROADCAST(m) */ * FROM duzy d JOIN maly m ON ...`

Źródło: Apache Spark SQL Performance Tuning – "Join Strategy Hints for SQL Queries", 2024.

26. Studia decyzyjne dla inżynierów

> **Scenariusz A: Złączenie z tabelą klientów trwa 5 godzin i kończy się błędem OOM**
> Twój zespół łączy tabelę kliknięć ze strony WWW (5 miliardów wierszy, 2 TB) z tabelą zarejestrowanych profili użytkowników (50 milionów wierszy, 15 GB) po kolumnie `user_id`. W Spark UI widzisz strategię `SortMergeJoin`, w której 199 tasków kończy się w 3 minuty, a 1 task ma rozmiar 450 GB danych Shuffle i wybucha błędem `OutOfMemoryError`. Analityk biznesowy mówi: "*Przecież 75% ludzi na stronie przegląda sklep bez logowania jako goście!*".
> 👉 **Pytanie inżynierskie:** Co znajduje się w kolumnie `user_id` dla gości? Dlaczego nawet automatyczny AQE Skew Join miał tu ciężkie życie i jak naprawisz ten potok w 3 linijkach kodu, skracając czas wykonania z 5 godzin do 4 minut?

Odpowiedź A: Goście niezalogowani mają `user_id = NULL` (lub `-1` / `"GUEST"`). Ponieważ stanowią 75% ruchu (1,5 TB!), wszystkie te wiersze mają identyczny hash i lądują w jednym tasku. Co więcej, gość niezalogowany NIGDY nie ma profilu w tabeli zarejestrowanych klientów! Zatem przesyłanie 1,5 TB przez sieć do JOIN-a jest całkowicie bezcelowe. Rozwiązanie: odfiltrować gości PRZED złączeniem (`df_logged = df.filter("user_id IS NOT NULL")`), wykonać błyskawiczny JOIN tylko na 25% zalogowanych, a gości dokleić na końcu przez `unionByName`!

27. Studia decyzyjne dla inżynierów

> **Scenariusz B: Kosztowny potok ML Feature Engineering w PySparku**
> Data Scientist napisał skrypt w PySparku, który przygotowuje cechy dla modelu kredytowego na 500 GB danych. Skrypt zawiera 12 wywołań klasycznych funkcji `@udf` w Pythonie (formatujących daty i wyliczających logarytmy), po każdym kroku wywołuje `df.cache()` oraz `print(df.count())` do logów, a na samym końcu zapisuje wynik przez `df.coalesce(1).write.parquet()`. Zadanie mieli na 64-rdzeniowym klastrze przez 6 godzin.
> 👉 **Pytanie inżynierskie:** Wymień wszystkie cztery kardynalne błędy architektoniczne w tym skrypcie i zaproponuj plan refaktoryzacji.

Odpowiedź B: 1. Zastąpić 12 klasycznych `@udf` wbudowanymi funkcjami `pyspark.sql.functions` (np. `F.log`, `F.date_format`) lub wektoryzowanymi Pandas UDF (Arrow). 2. Usunąć wszystkie pośrednie `print(df.count())`, które zmuszały klaster do 12-krotnego uruchamiania osobnych Jobów. 3. Usunąć zbędne wywołania `df.cache()` w liniowym potoku, które zapychały pamięć i wywoływały Disk Spill. 4. Usunąć zabójcze `coalesce(1)`, które dławiło cały ostatni etap do jednego rdzenia CPU!

28. Złote zasady inżyniera Apache Spark

1. **Nigdy nie walcz z optymalizatorem – pisz deklaratywnie:**
   - Używaj DataFrame API, Spark SQL i wbudowanych funkcji `pyspark.sql.functions`. Unikaj niskopoziomowych RDD oraz klasycznych funkcji UDF w Pythonie jak ognia.
2. **Traktuj operację Shuffle jak podatek od luksusu:**
   - Każdy `groupBy`, `join`, `distinct` i `repartition` to kosztowny zapis dyskowy i transfer sieciowy. Filtruj wiersze (`filter`) i odrzucaj zbędne kolumny (`select`) tak wcześnie, jak to możliwe, a małe tabele zawsze rozgłaszaj przez **BroadcastJoin**.
3. **Chroń Drivera przed przepełnieniem pamięci:**
   - Nigdy nie wywołuj `df.collect()` ani `df.toPandas()` na niezagregowanych tabelach produkcyjnych. Driver służy do dyrygowania klastrem, a nie do magazynowania terabajtów danych.
4. **Projektuj potoki z myślą o skośności danych (Data Skew):**
   - Zawsze zakładaj, że w danych produkcyjnych pojawią się klucze `NULL` oraz gorące klucze (*Hot Keys*). Korzystaj z **Adaptive Query Execution (AQE)**, izoluj wartości puste przed złączeniami i diagnozuj rozkład percentyli w **Spark UI**.

Podsumowanie Wykładu 4

- **Architektura i In-Memory DAG:** Apache Spark zastąpił dyskowy model MapReduce acyklicznym grafem wykonania (**DAG**) w pamięci RAM, zapewniając odporność na awarie poprzez logiczny graf rodowodu (**Lineage Graph**), a nie kosztowną replikację pamięci.
- **Model wykonania:** Praca w klastrze dzieli się według hierarchii **Application ➔ Job (Action) ➔ Stage (Shuffle Boundary) ➔ Task (1 Partycja = 1 Wątek CPU)**, przy czym transformacje wąskie (*Narrow*) wykonują się lokalnie w RAM, a szerokie (*Wide*) wymuszają wymianę sieciową **Shuffle**.
- **Patologie produkcyjne:** Trzy główne przyczyny awarii i spowolnień to narzut **Shuffle** (*Disk Spill*), błędy przepełnienia pamięci **OOM** (na poziomie Drivera lub Executora) oraz skośność danych (**Data Skew** powodująca powstawanie maruderów – *Stragglers*).
- **Silnik pod maską:** Wydajność Sparka napędzają trzy filary: kompilator **Catalyst Optimizer** (optymalizacje logiczne i kosztowe), **Project Tungsten** (pamięć *Off-Heap* `UnsafeRow` i *Whole-Stage CodeGen*) oraz **Adaptive Query Execution (AQE)** dynamicznie korygujący plan w trakcie działania.
- **Przyszłość:** Warstwa wykonawcza ewoluuje od czystego JVM ku natywnym, wektoryzowanym silnikom w C++ i Rust (**Databricks Photon**, **Velox**, **DataFusion Comet**).

Dziękuję za uwagę!

Następny wykład (W05):

Chmurowe platformy danych i ekosystem Databricks

Fenomen Lakehouse w chmurze • Databricks vs Snowflake • Zarządzanie i bezpieczeństwo: Unity Catalog • Deklaratywne potoki Delta Live Tables (DLT) • Silnik Photon, MLflow i FinOps

Specyfikacje, materiały źródłowe i diagramy dostępne w repozytorium kursu.